

MAST32001 Computational Statistics - Lecture 1

Variance, algorithms, and floating point arithmetic

Klaus Nordhausen

University of Helsinki

WS 2026

Today

1. What is computational statistics?
2. Variance: definitions and properties
3. Three ways to compute sample variance
4. Active R comparison using `microbenchmark`
5. What the comparison teaches us
6. Floating point arithmetic in statistical computing

Computational statistics

Definition

Computational statistics is the part of statistics concerned with the design, implementation, analysis, and practical use of computational methods for statistical inference, modelling, and data analysis.

It connects:

- ▶ statistical theory;
- ▶ numerical algorithms;
- ▶ simulation;
- ▶ data structures;
- ▶ programming; and
- ▶ reproducible scientific computing.

Why computational statistics?

Many statistical problems cannot be solved only by writing down a formula.

We often need computation to:

- ▶ simulate from probability models;
- ▶ approximate expectations or probabilities;
- ▶ optimize likelihoods or loss functions;
- ▶ sample from posterior distributions;
- ▶ assess uncertainty; and
- ▶ validate methods with simulation.

Variance: definitions and properties

Variance of a random variable

Let X be a random variable with finite mean

$$\mu = E(X).$$

The **variance** of X is

$$\text{Var}(X) = E\left((X - \mu)^2\right).$$

It measures the expected squared distance from the mean.

Alternative definition

Using algebra,

$$\text{Var}(X) = E((X - \mu)^2) = E(X^2) - (E(X))^2.$$

So variance can be described as:

- ▶ the mean squared deviation from the mean; or
- ▶ the second moment minus the squared first moment.

Both definitions are mathematically equivalent.

Basic properties

If X has finite variance, then:

$$\text{Var}(X) \geq 0.$$

For constants a and b ,

$$\text{Var}(a + bX) = b^2 \text{Var}(X).$$

Adding a constant changes the location but not the variance.

Multiplying by a constant changes the scale and multiplies the variance by the square of that constant.

Variance and independence

If X and Y are independent and have finite variances, then

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y).$$

More generally,

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y).$$

Variance is therefore connected to covariance and dependence.

Population variance and sample variance

Given observations

$$x_1, \dots, x_n,$$

the empirical mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

The usual unbiased sample variance is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Why divide by $n - 1$?

The factor $n - 1$ gives the unbiased estimator of the population variance when the observations are independent and identically distributed.

If $X_1, \dots, X_n$ are iid with variance σ^2 , then

$$E(s^2) = \sigma^2.$$

The divisor $n - 1$ is often called the degrees of freedom correction.

Computational question

How should we compute

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

in R?

We will compare three approaches:

1. two-pass algorithm;
2. one-pass computational formula;
3. Welford's online algorithm.

At this stage, treat them simply as three mathematically motivated ways to obtain the same statistical quantity.

Three algorithms

Algorithm 1: two-pass variance

The two-pass algorithm first computes the mean and then computes squared deviations.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

This is the direct textbook formula.

Algorithm 2: one-pass computational formula

Using the identity

$$\sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n\bar{x}^2,$$

we can write

$$s^2 = \frac{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2}{n - 1}.$$

This expresses the variance using sums of x_i and x_i^2 .

Algorithm 3: Welford's online algorithm

Welford's algorithm processes the observations **one at a time**.

After observing $x_1, \dots, x_i$, we maintain only two quantities:

- ▶ m_i : the mean of the first i observations;
- ▶ $M_{2,i}$: the sum of squared deviations from that current mean,

$$M_{2,i} = \sum_{j=1}^i (x_j - m_i)^2.$$

We do not need to store all previous observations.

Welford: initialization

Before seeing any observations, initialize

$$m_0 = 0, \quad M_{2,0} = 0.$$

The value $m_0 = 0$ is only a convenient starting value. After the first update,

$$m_1 = x_1, \quad M_{2,1} = 0.$$

We also keep track of the number of observations processed.

Welford: updating the mean

Suppose m_{i-1} is the mean after $i - 1$ observations.

For the new observation x_i , first calculate

$$\delta_i = x_i - m_{i-1}.$$

Then update the mean:

$$m_i = m_{i-1} + \frac{\delta_i}{i}.$$

Thus m_i is exactly the mean of $x_1, \dots, x_i$.

Welford: updating M_2

After updating the mean, calculate

$$\delta_{2,i} = x_i - m_i.$$

Then update

$$M_{2,i} = M_{2,i-1} + \delta_i \delta_{2,i}.$$

Equivalently,

$$M_{2,i} = M_{2,i-1} + (x_i - m_{i-1})(x_i - m_i).$$

At the end,

$$s^2 = \frac{M_{2,n}}{n-1}.$$

Welford: what is being stored?

At step i , the state is simply

$$(i, m_i, M_{2,i}).$$

So the algorithm needs constant memory:

- ▶ i : how many observations have been seen;
- ▶ m_i : their current mean;
- ▶ $M_{2,i}$: their accumulated squared deviation.

This makes the method useful when observations arrive sequentially or the full data set cannot be kept in memory.

Comparing algorithms

The three algorithms differ in implementation style.

- ▶ The two-pass algorithm is close to the definition.
- ▶ The one-pass formula uses algebraic simplification.
- ▶ Welford's algorithm is recursive and can process observations sequentially.

We now compare them empirically in R.

Active part: variance comparison

R TASK: implement and compare

Work in pairs or small groups.

Use the seed

```
set.seed(20260812)
```

and let

```
n <- 1e5
```

1. Implement the **two-pass**, **one-pass**, and **Welford online** algorithms in R.
2. Simulate n observations from $N(0, 1)$.
3. Reset the same seed and simulate n observations from $N(10^{12}, 1)$.
4. For both data sets, compare your three results with R's `var()`.
5. Compare the computing times of all four methods.

Decide yourself how to perform the timing comparison.

R implementation and comparison

Three variance functions

```
var_two_pass <- function(x) {  
  n <- length(x)  
  xbar <- mean(x)  
  sum((x - xbar)^2) / (n - 1)  
}  
  
var_one_pass <- function(x) {  
  n <- length(x)  
  (sum(x^2) - sum(x)^2 / n) / (n - 1)  
}
```

Three variance functions

```
var_welford <- function(x) {  
  n <- length(x)  
  m <- 0  
  M2 <- 0  
  
  for (i in seq_along(x)) {  
    delta <- x[i] - m  
    m <- m + delta / i  
    delta2 <- x[i] - m  
    M2 <- M2 + delta * delta2  
  }  
  
  M2 / (n - 1)  
}
```

Data used in the comparison

```
n <- 1e5  
  
set.seed(20260812)  
x_ok <- rnorm(n, mean = 0, sd = 1)  
  
set.seed(20260812)  
x_bad <- rnorm(n, mean = 1e12, sd = 1)
```

Using the same seed makes the random deviations identical before the shift is added.

Results: $N(0, 1)$

```
c(  
  two_pass = var_two_pass(x_ok),  
  one_pass = var_one_pass(x_ok),  
  welford = var_welford(x_ok),  
  R_var = var(x_ok)  
)  
  
##  two_pass  one_pass  welford    R_var  
## 0.9976083 0.9976083 0.9976083 0.9976083
```

Results: shifted normal

```
c(  
  two_pass = var_two_pass(x_bad),  
  one_pass = var_one_pass(x_bad),  
  welford = var_welford(x_bad),  
  R_var = var(x_bad)  
)
```

```
##  two_pass  one_pass  welford    R_var  
## 0.9976082 0.0000000 0.9976109 0.9976082
```

Timing with microbenchmark

```
library(microbenchmark)
```

```
mb_ok <- microbenchmark(  
  two_pass = var_two_pass(x_ok),  
  one_pass = var_one_pass(x_ok),  
  welford = var_welford(x_ok),  
  R_var = var(x_ok),  
  times = 20  
)  
  
summary(mb_ok)[, c("expr", "mean", "median")]
```

##	expr	mean	median
## 1	two_pass	1013.4162	903.7010
## 2	one_pass	900.3260	921.8010
## 3	welford	12151.4211	11536.3510
## 4	R_var	663.0708	567.6505

Timing: shifted data

```
mb_bad <- microbenchmark(  
  two_pass = var_two_pass(x_bad),  
  one_pass = var_one_pass(x_bad),  
  welford = var_welford(x_bad),  
  R_var = var(x_bad),  
  times = 20  
)  
  
summary(mb_bad)[, c("expr", "mean", "median")]
```

##	expr	mean	median
## 1	two_pass	953.7460	863.501
## 2	one_pass	734.9759	659.951
## 3	welford	10480.8810	9788.751
## 4	R_var	503.2608	476.801

What did we observe?

For moderate scale data, all four methods should give nearly identical answers.

For the large-offset data, the methods may no longer agree.

The disagreement is not caused by a different statistical definition of variance.

It is caused by how real-number arithmetic is represented on a computer.

From variance to floating point arithmetic

The one-pass formula contains the subtraction

$$\sum x_i^2 - \frac{1}{n} \left(\sum x_i \right)^2.$$

For large-offset data, these two quantities can be enormous and nearly equal.

The variance is the small difference between them.

This motivates the next topic: **floating point arithmetic**.

Floating point arithmetic

What is floating point arithmetic?

Computers store many real numbers approximately.

A floating point number is represented roughly as

$$\pm d_0.d_1d_2 \dots d_p \times b^e,$$

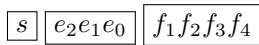
where:

- ▶ b is the base;
- ▶ p is the precision;
- ▶ e is the exponent.

Most statistical computing uses IEEE double precision.

A toy 8-bit floating point system

To see the idea, consider a simplified **8-bit** floating point system:



with

- ▶ 1 sign bit s ;
- ▶ 3 exponent bits;
- ▶ 4 fraction bits;
- ▶ exponent bias = 3;
- ▶ normalized value

$$(-1)^s (1.f_1f_2f_3f_4)_2 2^{E-3}.$$

This is a teaching example, not an IEEE standard format.

8-bit example

Consider

$$\boxed{0} \boxed{100} \boxed{1010}.$$

Here

$$s = 0, \quad E = (100)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 4, \quad E - 3 = 1.$$

The significand is

$$\begin{aligned}(1.1010)_2 &= 1 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4} \\ &= 1 + \frac{1}{2} + \frac{1}{8} = 1.625.\end{aligned}$$

Therefore the represented number is

$$1.625 \times 2^1 = \boxed{3.25}.$$

TASK: decode an 8-bit float

Using the same toy system,

$$(-1)^s (1.f_1 f_2 f_3 f_4)_2 2^{E-3},$$

what decimal number is represented by

$$\boxed{1} \boxed{101} \boxed{0110} ?$$

Work it out from:

1. the sign;
2. the biased exponent;
3. the binary significand.

Solution: 8-bit task

For

$$\boxed{1} \boxed{101} \boxed{0110},$$

the sign is negative and

$$E = (101)_2 = 5, \quad E - 3 = 2.$$

The significand is

$$(1.0110)_2 = 1 + \frac{1}{4} + \frac{1}{8} = 1.375.$$

Hence

$$-1.375 \times 2^2 = \boxed{-5.5}.$$

Double precision in R

R's usual numeric type is double precision floating point.

A double uses 64 bits:

- ▶ 1 sign bit;
- ▶ 11 exponent bits;
- ▶ 52 stored fraction bits;
- ▶ approximately 15-16 decimal digits of precision.

This is very accurate, but it is not exact real arithmetic.

First example in R

```
0.1 + 0.2
```

```
## [1] 0.3
```

```
0.1 + 0.2 == 0.3
```

```
## [1] FALSE
```

```
print(0.1 + 0.2, digits = 22)
```

```
## [1] 0.30000000000000000444089
```

```
print(0.3, digits = 22)
```

```
## [1] 0.29999999999999999888978
```

Machine epsilon

Machine epsilon is the distance between 1 and the next larger representable floating point number.

```
.Machine$double.eps
```

```
## [1] 2.220446e-16
```

```
1 + .Machine$double.eps
```

```
## [1] 1
```

```
1 + .Machine$double.eps / 2
```

```
## [1] 1
```

Small changes can disappear when they are too small relative to the number being stored.

Relative precision

Floating point precision is relative.

Around 1, double precision is extremely fine.

Around 10^{16} , the spacing between adjacent representable numbers is much larger.

```
x <- c(1, 1e8, 1e12, 1e16)
data.frame(
  x = x,
  next_value = x + 1,
  changed_by_adding_1 = (x + 1) != x
)
```

##		x	next_value	changed_by_adding_1
##	1	1e+00	2e+00	TRUE
##	2	1e+08	1e+08	TRUE
##	3	1e+12	1e+12	TRUE
##	4	1e+16	1e+16	FALSE

Rounding error

Most arithmetic operations are rounded to the nearest representable floating point number.

This means that every operation may introduce a tiny error.

Usually these errors are harmless.

They become important when the computation amplifies them.

Cancellation

Cancellation occurs when nearly equal numbers are subtracted.

If many leading digits cancel, the remaining result may contain much less accurate information than expected.

This is the mechanism behind the variance example.

Cancellation in sample variance

The formula

$$\frac{\sum x_i^2 - (\sum x_i)^2/n}{n - 1}$$

can subtract two huge, nearly equal quantities.

This is why the one-pass formula may be unreliable for data with a large offset and small spread.

Overflow

Overflow occurs when the result is too large to represent.

```
1e308
```

```
## [1] 1e+308
```

```
1e308 * 10
```

```
## [1] Inf
```

In statistics this can happen with likelihoods, densities, exponentials, and combinatorial quantities.

Underflow

Underflow occurs when the result is too close to zero to represent accurately.

```
1e-315
```

```
## [1] 1e-315
```

```
1e-315 * 1e-2
```

```
## [1] 1e-317
```

```
1e-315 * 1e-2 * 1e-8
```

```
## [1] 0
```

Underflow is common when multiplying many small probabilities.

Likelihoods

For independent observations, the likelihood is often a product:

$$L(\theta) = \prod_{i=1}^n f(x_i; \theta).$$

Products of many densities or probabilities can become extremely small.

Therefore we usually work with the log-likelihood:

$$\ell(\theta) = \sum_{i=1}^n \log f(x_i; \theta).$$

Product versus log-likelihood

```
p <- rep(0.01, 1000)
```

```
prod(p)
```

```
## [1] 0
```

```
sum(log(p))
```

```
## [1] -4605.17
```

The product underflows to zero, but the log-likelihood is still finite.

Tail probabilities

Extreme tail probabilities are numerically delicate.

For a standard normal variable Z ,

$$P(Z > x) = 1 - \Phi(x).$$

Direct subtraction from 1 may lose accuracy when $\Phi(x)$ is very close to 1.

Tail-specific functions

```
x_tail <- c(2, 5, 8, 10)

data.frame(
  x = x_tail,
  one_minus_pnorm = 1 - pnorm(x_tail),
  pnorm_upper_tail = pnorm(x_tail, lower.tail = FALSE),
  log_upper_tail = pnorm(x_tail, lower.tail = FALSE, log.p = TRUE)
)
```

```
##      x one_minus_pnorm pnorm_upper_tail log_upper_tail
## 1  2      2.275013e-02      2.275013e-02      -3.783184
## 2  5      2.866516e-07      2.866516e-07     -15.064998
## 3  8      6.661338e-16      6.220961e-16     -35.013437
## 4 10      0.000000e+00      7.619853e-24     -53.231285
```

Use `lower.tail = FALSE` instead of `1 - pnorm(x)` for upper tails.

Use `log.p = TRUE` for very small probabilities.

Log-sum-exp

A common problem is computing

$$\log \left(\sum_i \exp(a_i) \right)$$

when some a_i are very large or very small.

The stable identity is

$$\log \left(\sum_i \exp(a_i) \right) = m + \log \left(\sum_i \exp(a_i - m) \right),$$

where $m = \max_i a_i$.

Log-sum-exp in R

```
log_sum_exp <- function(a) {  
  m <- max(a)  
  m + log(sum(exp(a - m)))  
}
```

```
a <- c(1000, 1001, 1002)
```

```
log(sum(exp(a)))
```

```
## [1] Inf
```

```
log_sum_exp(a)
```

```
## [1] 1002.408
```

Summation order

If numbers of very different sizes are added together, small terms may not change the running sum.

```
sum_naive <- function(x) {  
  s <- 0  
  for (xi in x) {s <- s + xi}  
  s  
}  
x <- c(1e16, rep(1, 1000), -1e16)  
sum_naive(x)
```

```
## [1] 0
```

```
sum_naive(rev(x))
```

```
## [1] 0
```

```
sum_naive(c(rep(1, 1000), 1e16, -1e16))
```

```
## [1] 1000
```

The mathematical sum is 1000, but floating point summation can depend on order.

Consequence for statistics

Floating point arithmetic can affect:

- ▶ means and variances;
- ▶ covariance and correlation matrices;
- ▶ likelihoods and posterior densities;
- ▶ p-values and confidence intervals;
- ▶ simulation studies;
- ▶ optimization;
- ▶ matrix decompositions;
- ▶ MCMC acceptance probabilities.

Practical warning signs

Be careful when you see:

- ▶ very large values combined with very small variability;
- ▶ probabilities close to 0 or 1;
- ▶ subtraction of similar quantities;
- ▶ exponentials of large numbers;
- ▶ products of many probabilities;
- ▶ nearly singular matrices;
- ▶ warnings such as NaN, Inf, or “computationally singular”.

Good computational habits

Use stable algorithms and built-in functions.

Prefer:

- ▶ `var(x)` over a hand-written one-pass variance formula;
- ▶ `pnorm(x, lower.tail = FALSE)` over `1 - pnorm(x)`;
- ▶ log-likelihoods over likelihood products;
- ▶ `log.p = TRUE` for tiny probabilities;
- ▶ QR or SVD methods for difficult least-squares problems;
- ▶ explicit numerical checks in simulation studies.

Statistical and numerical error

In computational statistics we often face two kinds of error:

1. **Statistical error**: error due to finite data or finite simulation.
2. **Numerical error**: error due to finite precision computation or approximation.

Good computational statistics tries to understand and control both.

Summary

What you should remember

- ▶ Computational statistics connects statistical ideas with algorithms and implementation.
- ▶ Variance has several equivalent mathematical definitions.
- ▶ Equivalent mathematical formulas can lead to different algorithms.
- ▶ The active variance comparison shows why implementation matters.
- ▶ Floating point arithmetic is approximate, not real-number arithmetic.
- ▶ Stable statistical computing often means using log scales, tail-specific functions, and numerically stable algorithms.